

HERNANDO, PANTINE B.

PICO CTF-CHALLENGE 6

BSIT-3

Packets Primer

Medium Forensics picoCTF 2022 pcap

AUTHOR: LT 'SYREAL' JONES

Description

Download the packet capture file and use packet analysis software to find the flag.

- Download packet capture

Hints

1

Wireshark, if you can install and use it, is probably the most beginner friendly packet analysis software product.

30,007 users solved



89%

Liked



picoCTF{FLAG}

Submit Flag

1. CHALLENGE OVERVIEW

The objective of this challenge was to extract hidden a flag from a network traffic capture filed named **network-dump.flag.pcap**. the file contains a small sequence of TCP packets representing a brief communication between two hosts.

2. Identification and analysis



No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000		10.0.2.15	10.0.2.4	TCP	74	48750 → 9000 [SYN] Seq=0 Win=64240 Len=0 MSS=1
2	0.000896		0.000896 10.0.2.4	10.0.2.15	TCP	74	9000 → 48750 [SYN, ACK] Seq=0 Ack=1 Win=65160
3	0.001006		0.000110 10.0.2.15	10.0.2.4	TCP	66	48750 → 9000 [ACK] Seq=1 Ack=1 Win=64256 Len=0
4	0.001225		0.000219 10.0.2.15	10.0.2.4	TCP	126	48750 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=64256
5	0.002031		0.000806 10.0.2.4	10.0.2.15	TCP	66	9000 → 48750 [ACK] Seq=1 Ack=61 Win=65152 Len=0

Frame 4: Packet, 126 bytes on wire (1008 bits), 126 bytes captured (1008 bits) on interface 0

Ethernet II, Src: PCSSystemtec_af:39:9f (08:00:27:af:39:9f), Dst: PCSSystemtec_08:00:27:af:39:9f (08:00:27:af:39:9f)

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 10.0.2.4

Transmission Control Protocol, Src Port: 48750, Dst Port: 9000, Seq: 1, Ack: 1, Win: 64256, Len: 0

Data (60 bytes)

0000 08 00 27 93 ce 73 08 00 27 af 39 9f 08 00 45 00 ... s : 9 : E

0010 00 70 50 c2 40 00 40 06 d1 b3 0a 00 02 0f 0a 00 ... pP @ @ :

0020 02 04 be 6e 23 28 27 ec d4 b7 bd 26 99 bc 80 18 ... n# (' &

0030 01 f6 18 75 00 00 01 01 08 0a 8d cf e9 65 68 f0 ... u : eh

0040 f1 c3 70 20 69 20 63 20 6f 20 43 20 54 20 46 20 ... p i c o C T F

0050 7b 20 70 20 34 20 63 20 6b 20 33 20 37 20 5f 20 ... { p 4 c k 3 7 _

0060 35 20 68 20 34 20 72 20 6b 20 5f 20 63 20 65 20 ... S h 4 r k _ c e

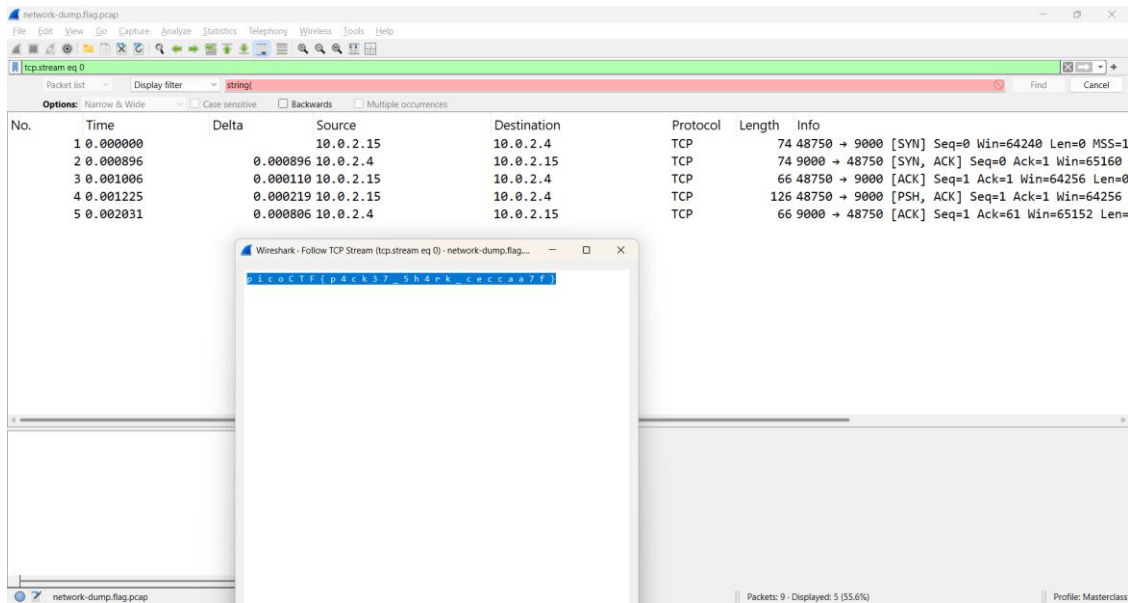
0070 63 20 63 20 61 20 61 20 37 20 66 20 7d 0a ... c c a a 7 f }

- Initial inspection: the raw packet data showed a TCP handshake followed by data containing the string "picoCTF" with spaces between each character.
- Protocol: the traffic uses the

TRANSMISSION CONTROL PROTOCOL (TCP).

- Key Packet Frame 4 (a PSH, ACK packet) was identified as the primary carrier of the flag payload.

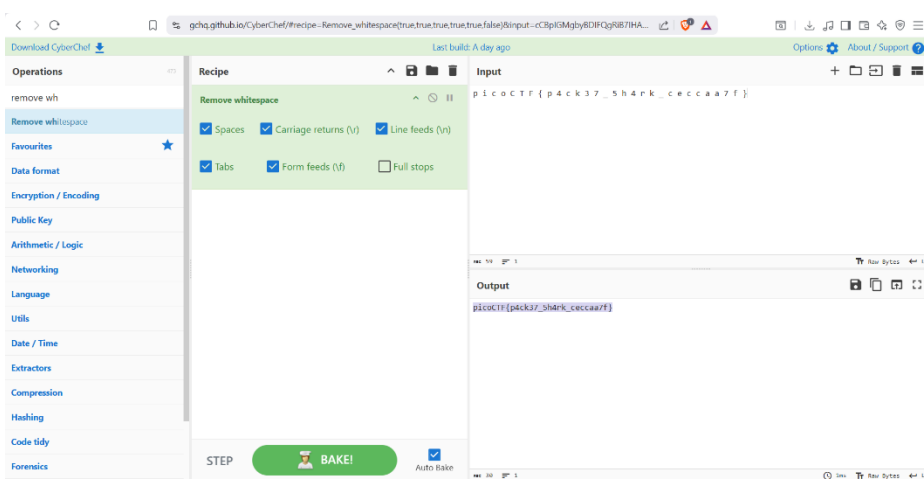
- Data Structure: the flag was transmitted with spaces between each character: p i c o C T F { p 4 c k 3 7 _ 5 h 4 r k _ c e c c a a 7 f }.



3. Resolution reconstruction:

used the **Follow TCP Stream** feature in Wireshark to aggregate the fragmented data into a single readable string.

- **Packet Reconstruction:** Used the **Follow TCP Stream** feature in Wireshark to aggregate the fragmented data into a single readable string.
- **Data Sanitization:** Copied the spaced-out string into **CyberChef**.
- **Recipe Applied:** Utilized the **Remove whitespace** operation to strip all spaces, carriage returns, and line feeds.
- **Final Flag Extraction:** The cleaned output revealed the valid flag format.
- **Final Flag:** picoCTF{p4ck37_5h4rk_ccecaa7f}



- **Short Reflection**
This challenge highlighted the importance of looking beyond individual packets to understand the "stream" of data. While the flag was technically visible in the raw hex dump, using tools like **Wireshark's Follow Stream** and **CyberChef's Remove whitespace** turned a manual, error-prone extraction into a quick and accurate process. It served as a great reminder

that data in transit is often formatted differently than its final intended state.